

Inter-process Communication

IPC

- One process, sometimes, require the output of other process
- Therefore, there is a need of well structured communication among processes.
- Not preferring interrupts to draw attention.

IPC

- Three issues in IPC-
 1. How one process can pass information to another
 2. Making sure that two or more processes do not get into each other's way when engaging in critical activities (both wants last 1 MB space of virtual memory)
 3. Proper sequencing when dependency is present (if A produces data that B prints, then B cannot print unless A is producing some data)

Inter-thread Communication

- In case of ITC, the same issues are concerns.
- The first one is not a headache as threads share some common resources and address space; so, they can easily communicate
- But the other twos are issues in ITC as well.

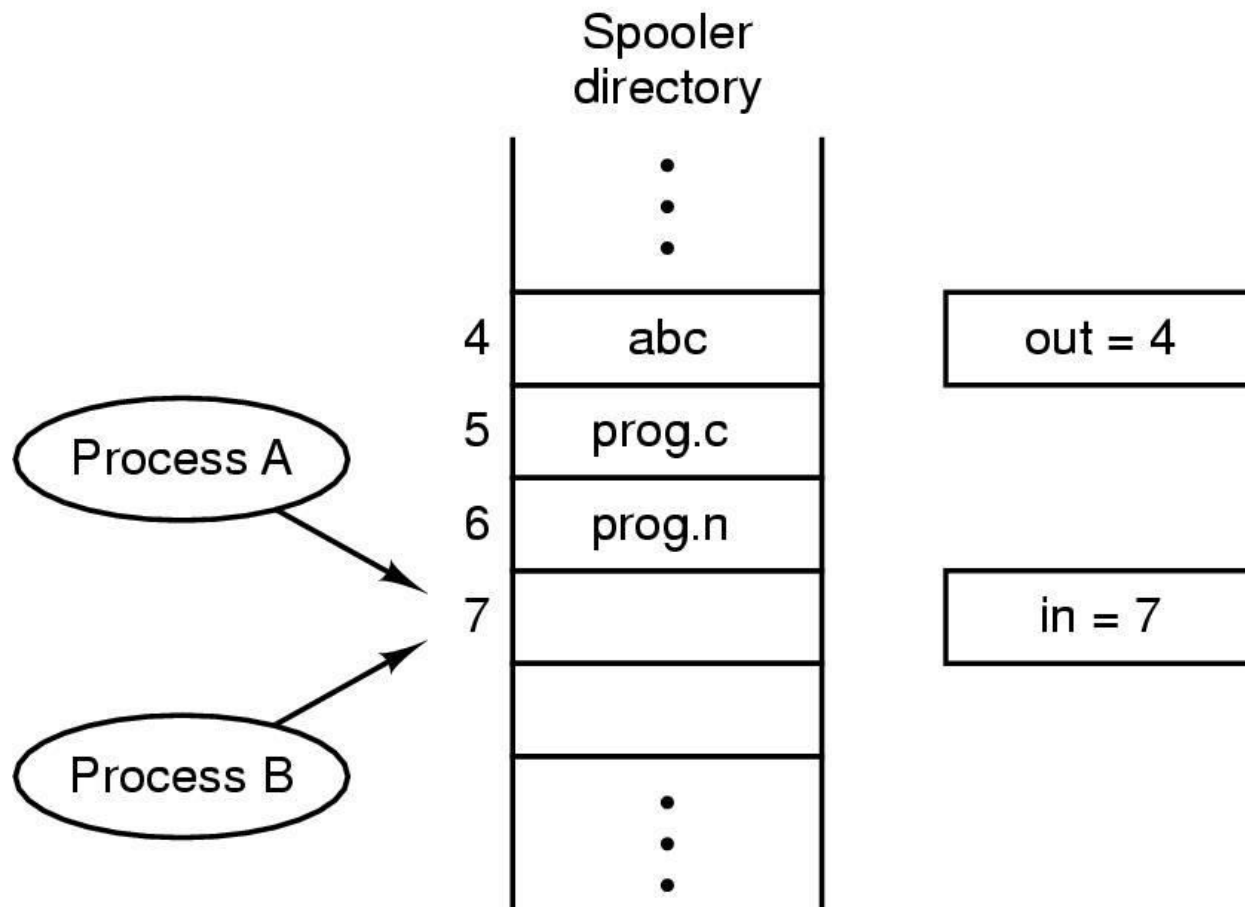
Race Condition

- A process wants to print a file.
- It enters the file name in a special **printer directory**
- The **Printer Daemon** periodically checks to see if there is any file to print
- If any file is there the Printer Daemon prints them and removes their names from the directory

Race Condition

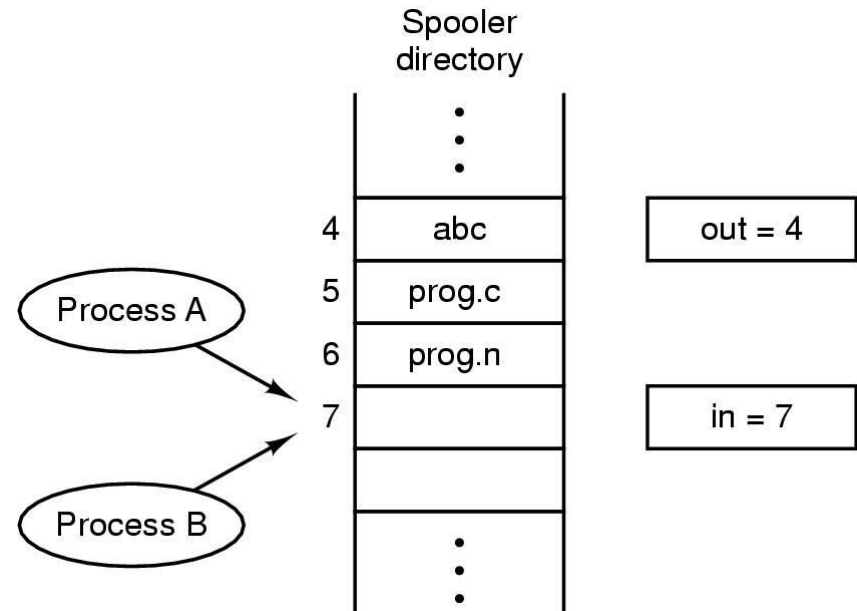
- Imagine our directory has a very large number of slots (numbered 0,1,2,...) and each one can hold a file name
- There are two shared variables- *out* that points to the next file to be printed and *in* that points to the next free slot in the directory

Race Condition



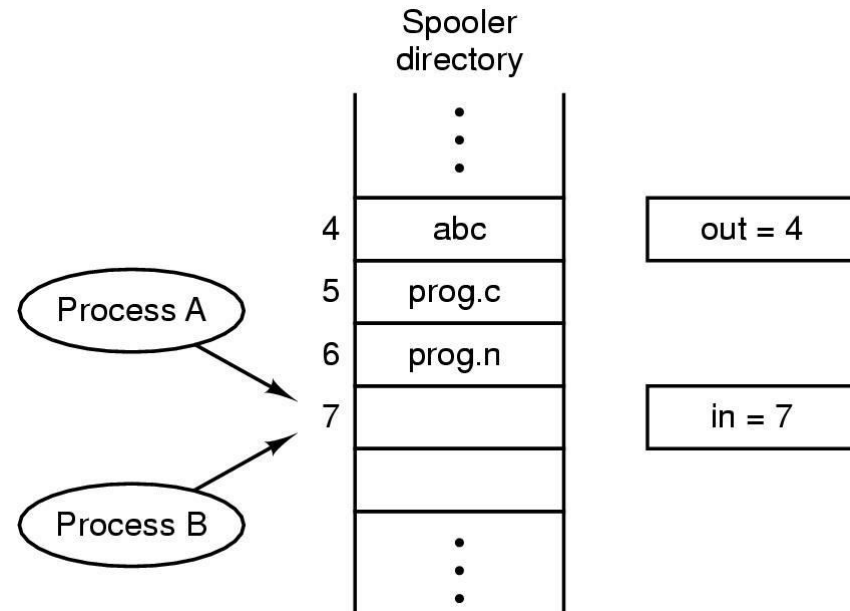
Race Condition

- A reads *in* and stores the values 7 to its local variable
- Then a context switch from A to B occurs
- B also reads *in* and stores the value 7 to its local variable
- B continues to run and it stores the name of its file in slot 7 and updates *in* to 8.
- Then it goes off and does other things



Race Condition

- A runs again starting from the place it left off
- It looks its local variable and finds 7 there and writes the file name in slot 7
- Then A sets *in* to 8
- As everything went fine, the printer daemon will not raise any error



Race Condition

- Process B never gets the chance
- Situations like this where two or more processes are reading or writing some shared data and the final result depends on who ran precisely are called **race conditions**

Critical Regions

- How can we avoid race conditions?
- One way to avoid that is prohibiting more than one process from reading and writing the shared data at the same time
- This is called **Mutual Exclusion**

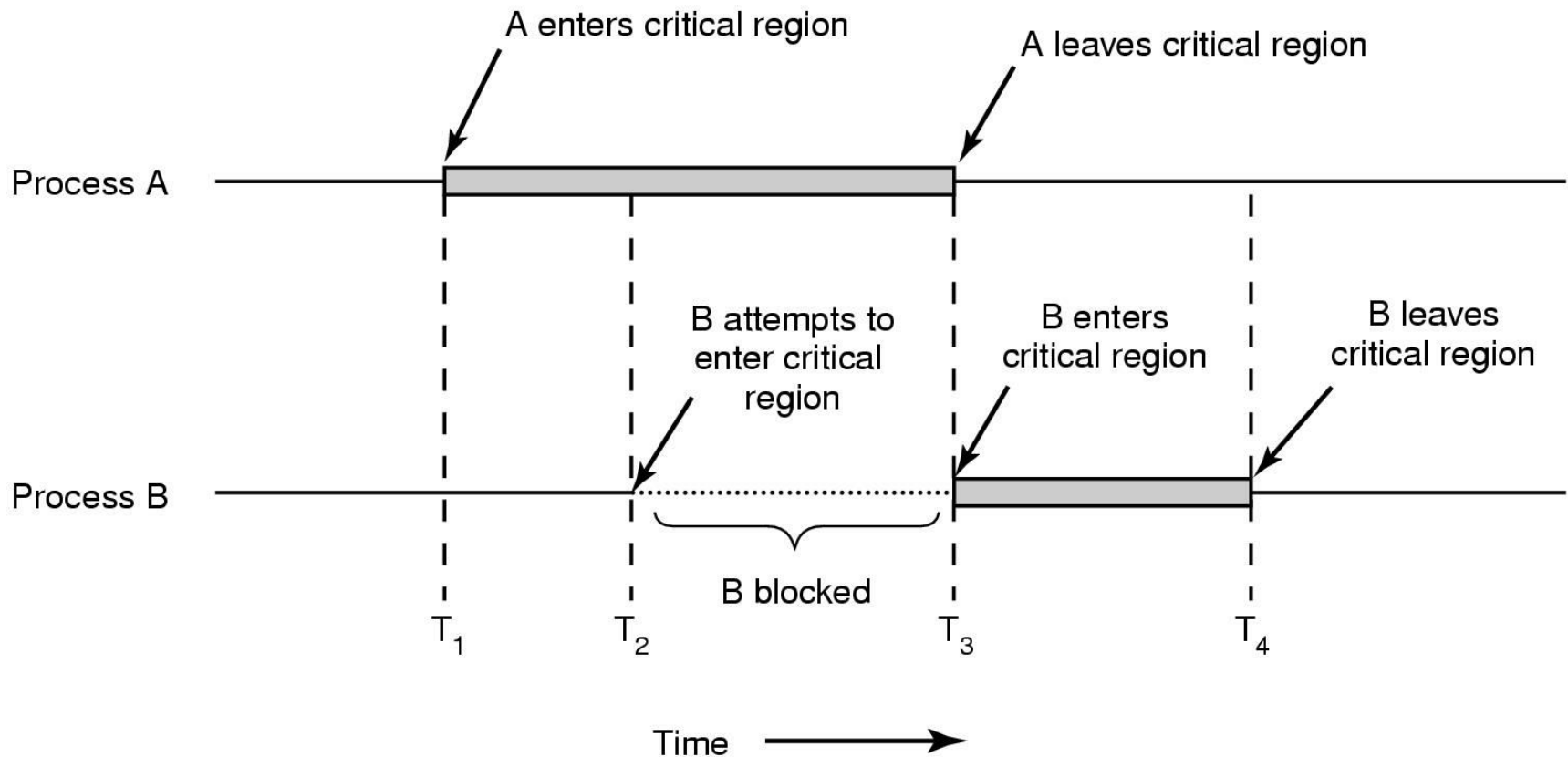
Critical Regions

- On the other hand, let's think in abstract way.
- A process is busy doing internal computations and other things that do not lead race conditions
- And sometimes it is busy to access shared memory and files or in doing other critical things that lead race conditions
- The part of the program where shared memory or resources are accesses is called **Critical Regions**

Critical Regions

- We need four conditions to hold for a good solution with mutual exclusion-
 1. No two processes simultaneously in critical region
 2. No assumptions made about speeds or numbers of CPUs
 3. No process running outside its critical region may block another process
 4. No process must wait forever to enter its critical region

Mutual Exclusion with Critical Regions



How can we achieve mutual exclusion?

- Now, let's examine various proposals to achieve mutual exclusion
- While one process is busy to update shared memory in its critical region, no other process will enter its critical region and cause trouble

Disabling Interrupts

- When a process enters into its critical region, it disables all interrupts
- While leaving its critical region, it re-enables all interrupts
- Unattractive and unwise to give user processes the power of turning off interrupts. What if one of them did it and never turned them on again!! ☹
- That is the end of the system!!! ☹
- It is often useful technique within the OS itself but not suitable as a general mutual exclusion mechanism

Lock Variables

- It's a software solution
- When a process wants to enter into the critical region, it checks the lock variable
- If it is 0, the process sets it to 1 and enters into its critical region
- If it is 1, the process waits

Lock Variables

- One process reads the lock and sees 0
- Before it sets 1, another process is scheduled, runs and sets the lock 1
- When the first process runs, it will also set the lock 1
- Two processes will be in their critical regions in the same time.

Common Problem

So far, the techniques we learnt (except disabling interrupts)-

1. Lock variables
 2. Strict Alternation
 3. Peterson's Solution
- To achieve mutual exclusion, all have a common problem- Busy Waiting
 - In case of prioritized scheduling, low prioritized processes will never be fed if Busy Waiting takes place

Different Mechanisms with Sleep and Wake

- Now, we will see more mechanisms to achieve mutual exclusion
- These techniques will use **Sleep** and **Wake**- two system calls
- Sleep causes a process to be suspended until another process wakes it up
- Wake causes a process to wake up

The Producer-Consumer Problem

- When the producer sees a full buffer and goes to sleep. When the consumer takes out an item, it awakes the producer
- When the consumer sees an empty buffer and goes to sleep. When the producer puts an item, it awakes the consumer

The Producer-Consumer Problem

- We will use count as a variable to stop race conditions
- If the maximum no of information the buffer stores is N , then producer first checks if $\text{count} = N$. If yes, then it sleeps; otherwise it will add an item and increment count by 1
- The consumer tests count. If $\text{count} = 0$, then it sleeps; otherwise it removes an information and decrements count by 1

The Producer-Consumer Problem

```
#define N 100
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

/* number of slots in the buffer */
/* number of items in the buffer */

/* repeat forever */
/* generate next item */
/* if buffer is full, go to sleep */
/* put item in buffer */
/* increment count of items in buffer */
/* was buffer empty? */

/* repeat forever */
/* if buffer is empty, got to sleep */
/* take item out of buffer */
/* decrement count of items in buffer */
/* was buffer full? */
/* print item */

The Producer-Consumer Problem

- Two processes share a common, fixed-sized buffer
- The producer puts information into the buffer
- The consumer takes it out

Problem arises when-

1. Producer wants to put information into a buffer that is full
2. Consumer wants to get information from a buffer that is empty

The Producer-Consumer Problem

- The buffer is empty; the consumer is about to read
 $\text{count} = 0$
- Scheduler decides **at that very instant** to stop consumer and start producer
- The producer inserts an item and increases count by 1
- The producer will wake the consumer up
- The consumer was not logically sleeping. So, wake signal is lost.
- The consumer, on its next run, sees $\text{count} = 0$ and sleeps

The Producer-Consumer Problem

- Soon, the producer fills up the buffer and also goes to sleep
Both will sleep forever
- The main problem here is the lost wake up signal.
- If it were not lost, everything would have worked
- To solve this problem, we can use **wake up waiting bit**
- When a wake up is sent to a process (producer or consumer) that is not sleeping, this bit will be set.
- When the sender goes to sleep, it checks this bit
If it is on, then the process will not sleep itself (because someone MAYBE sleeping)

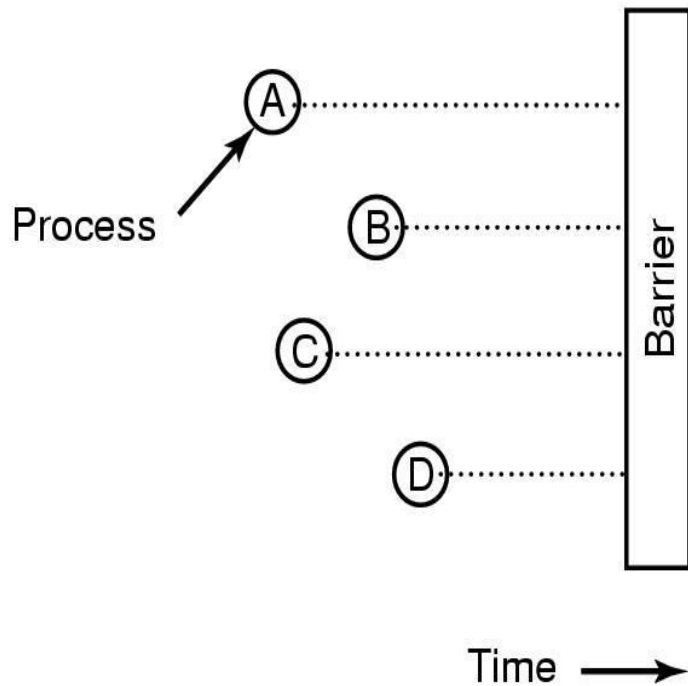
Barriers

Some applications are divided into phases

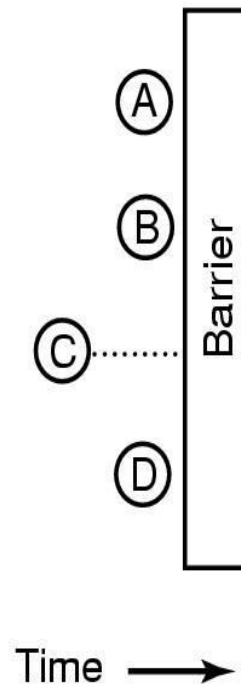
- And have the rule that no process may proceed to the next phase until all processes are ready to proceed to the next phase.
- This behaviour maybe achieved by placing a Barrier at the end of each phase.

When a process reaches the barrier, it is blocked until all processes reach the barrier.

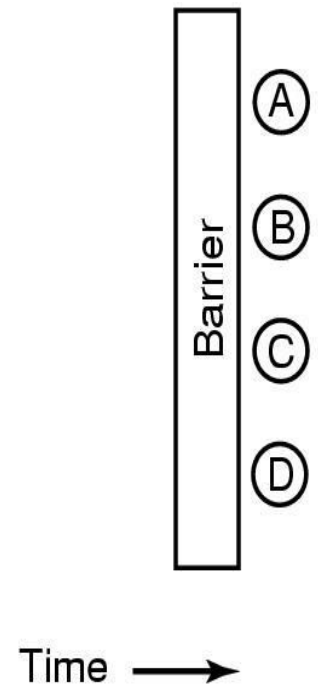
Barriers



(a)



(b)



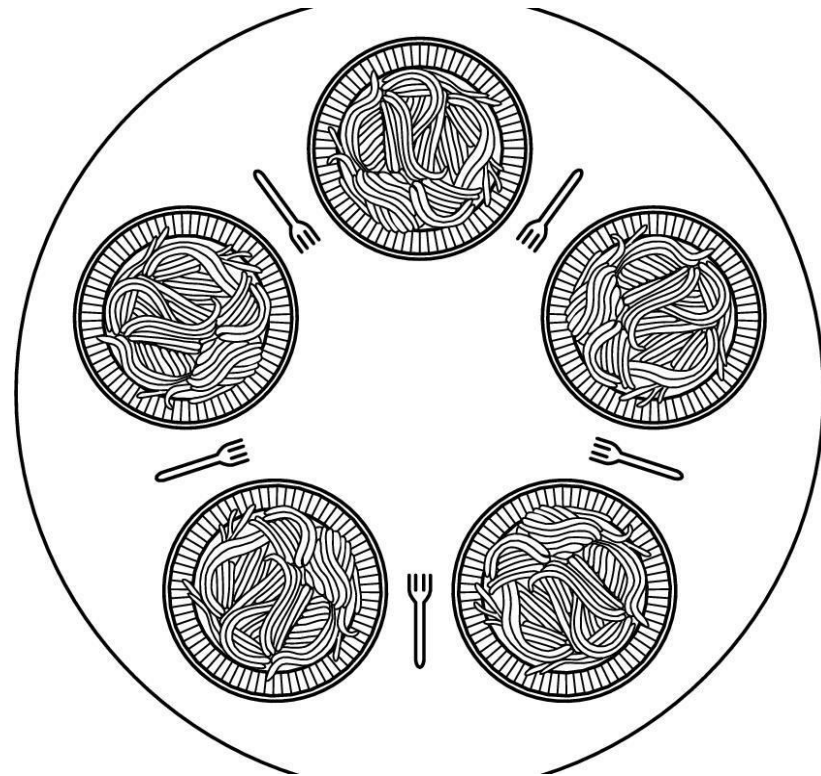
(c)

Classical IPC Problems



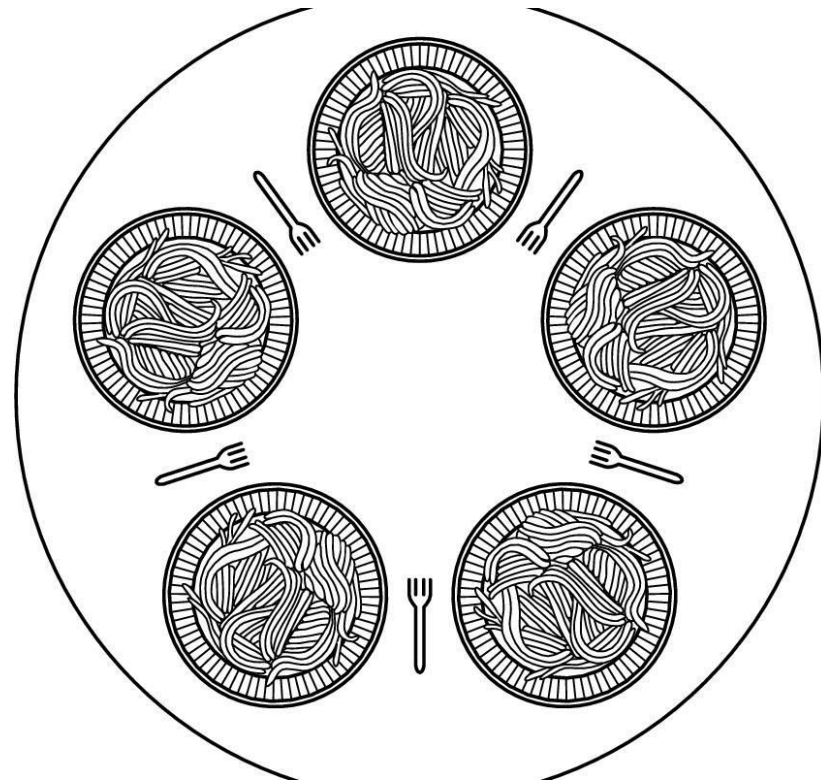
Dining Philosopher Problem

- Five Philosophers seated around the circular table
- Each has a plate of Spaghetti
- Each needs two forks to eat it
- Between each pair of plates there is one fork



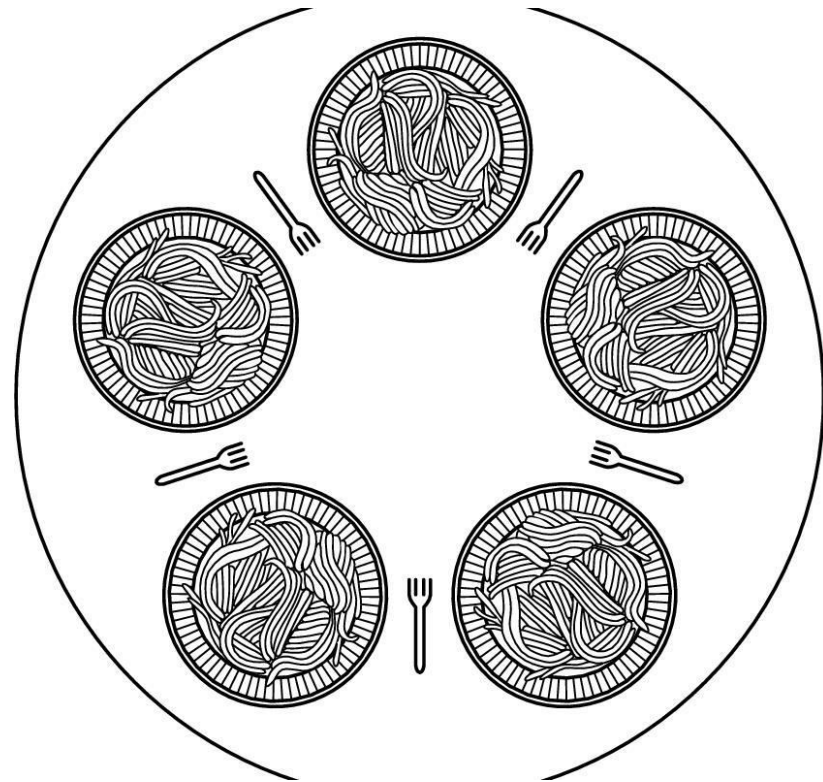
Dining Philosopher Problem

- The lives of the philosophers consist of two things- eat and think
- When a philosopher is hungry, he tries to acquire his left/ right fork, *one at a time, in either order*
- He only eats after successful acquisition of the forks
- He eats for a while and then puts them back to think



Dining Philosopher Problem

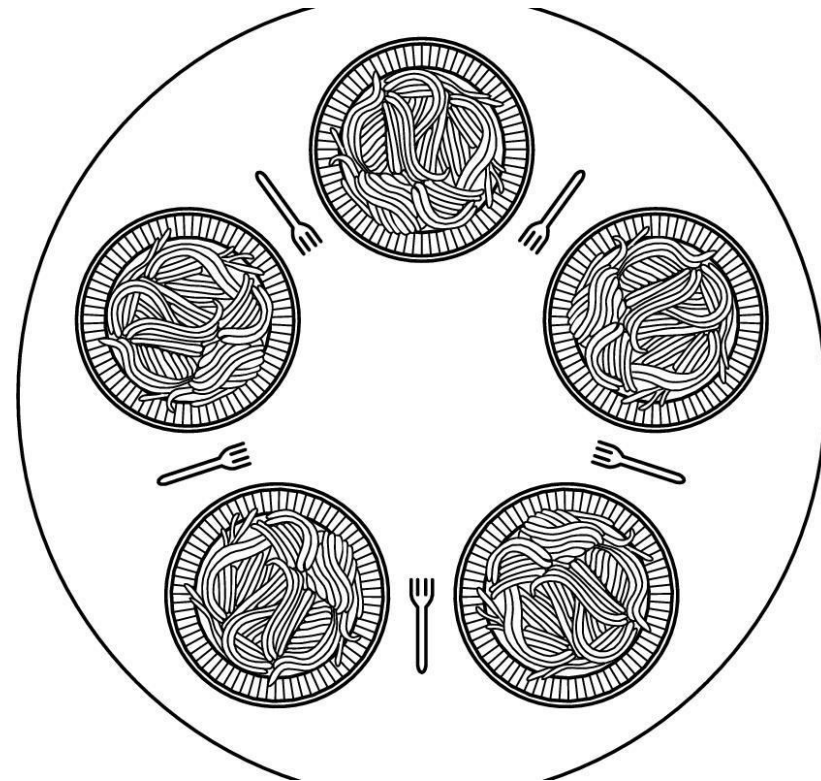
Is it possible to have a solution so that no philosophers will be annoyed? (when his turn is eating, he eats; when his turn is thinking, he thinks)



Solutions

- Philosopher will wait until its *desired* fork is available
- He will grab it when it's available
- What if all the five philosophers take their left forks simultaneously?

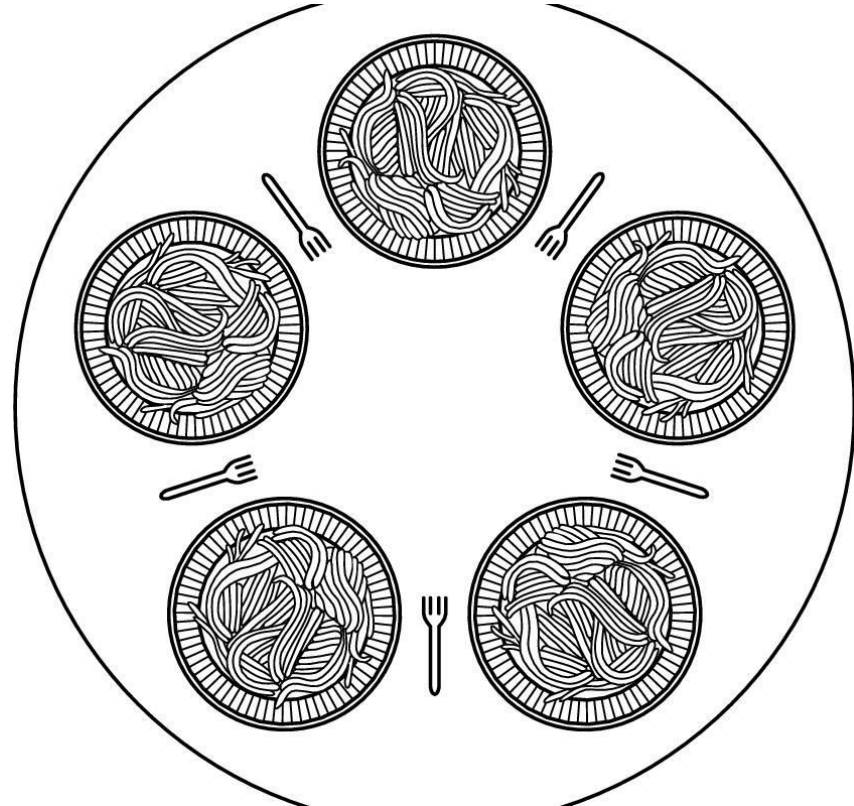
None will be able to take their right forks; there will be **deadlock**



Solutions

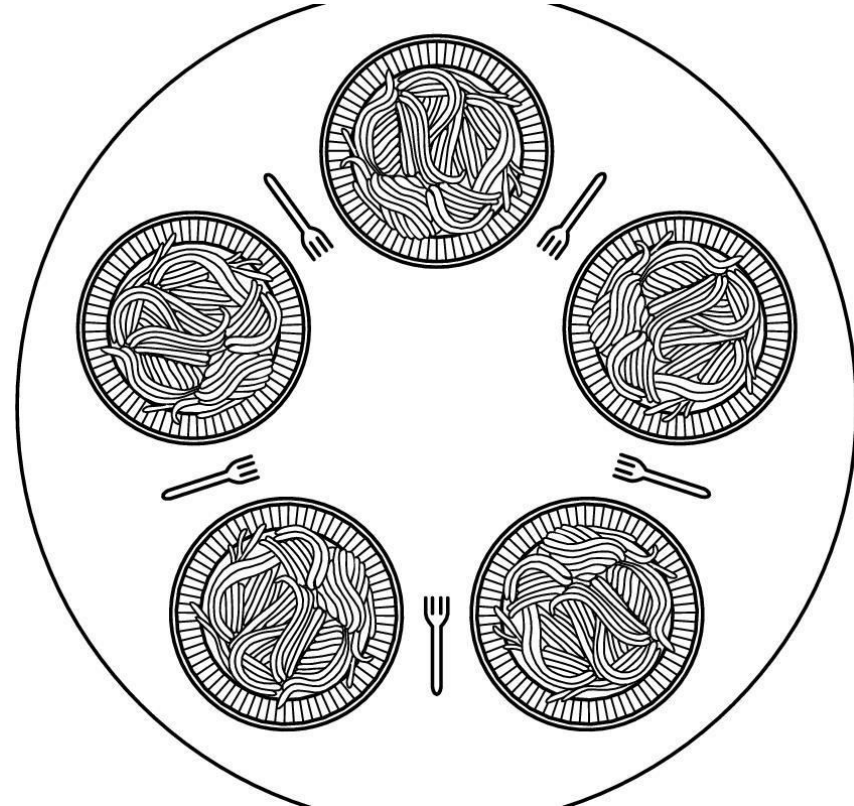
- After taking the left fork, philosopher will check if its right fork is available
- If it's not, philosopher puts back his left fork, wait for some times and proceeds again in similar way
- What if all philosophers start simultaneously?

They will never find their right fork available causing **starvation**



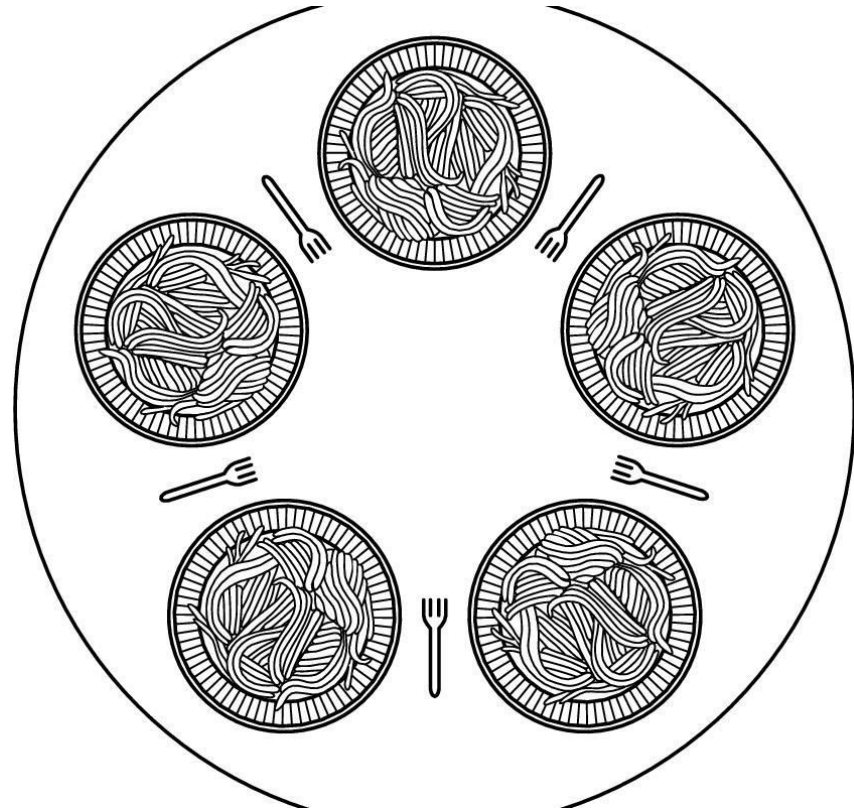
Solutions

- Using random start timer can solve this problem, but not ultimately
- Ethernet LAN works in this way, and this happens to be finer solution, but again, not the best; there is always a chance to have a failure



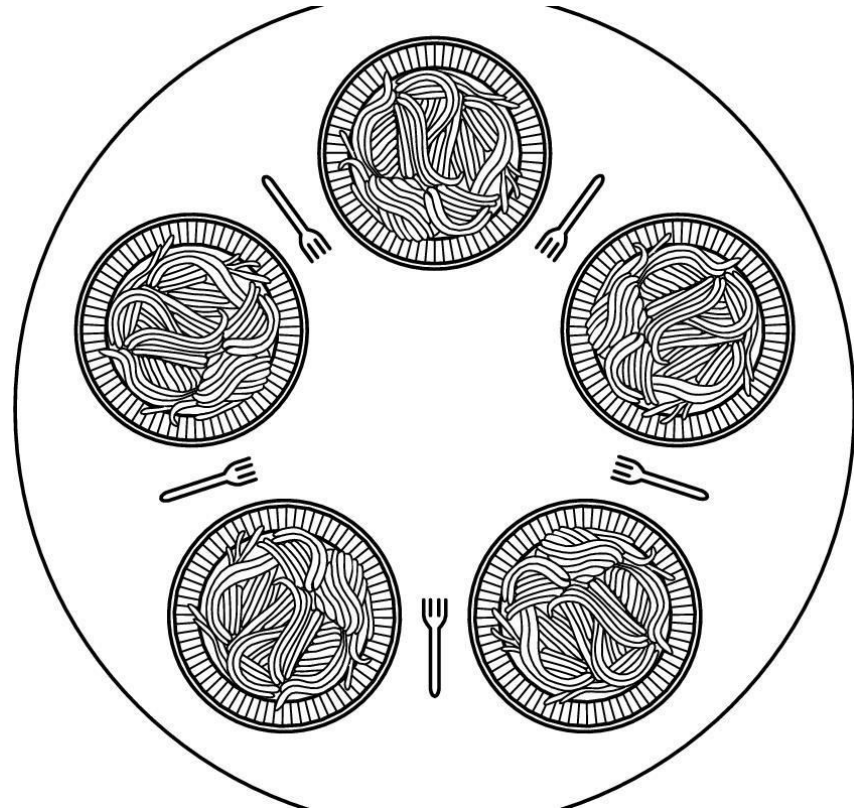
Solutions

- Well, there is a solution that will stop deadlock and starvation
- When philosopher wants to acquire a fork, she downs mutex; when she releases, she ups mutex
- The only drawback is only 1 of 5 philosophers can eat at a time though there is a best chance of 2 philosophers eating at same time



Solutions

- Our last solution will be deadlock free and achieve maximum parallelism.
- A philosopher will have 3 states- eating, thinking, or hungry (trying to acquire forks)
- A philosopher will move to eating state only if none of the neighbours is eating
- Only need is that each philosopher will have individual semaphores



The Readers-Writers Problem

- Dining philosopher problem defines the situation where processes compete for limited resources
- The Readers-Writers problem defines the situation where database access is required.
- A reader reads... the writer writes... but the reader is away and with an old value from the database: simply, this is the readers-writers problem

Solutions

- When a reader comes along, it UPs a semaphore-means, hey, we are reading, do not disturb
- If a writer comes, then it has to wait.
- If more readers come, they are allowed

Solution

- When an active reader is reading, then a writer comes.
- It sees that a reader is reading, the writer then waits
- If more reader comes, they are queued
- When the active reader finishes, the writer takes place its schedule
- After finishing of writer, the queued readers are given chances.
- Concurrency problem and lower performance is key issue here

The Sleeping Barber Problem

- In a barber shop, there is one barber, some chairs and some customers
- A barber sleeps if there is no customer (not even on chairs, waiting for a haircut ☹)
- A customer wakes up the barber if it's his turn to get his haircut
- A customer waits if there is any chair left
- A customer leaves, if all the chairs are occupied



```
#define CHAIRS 5
```

```
typedef int semaphore;
```

```
semaphore customers = 0;
```

```
semaphore barbers = 0;
```

```
semaphore mutex = 1;
```

```
int waiting = 0;
```

```
void barber(void)
```

```
{
```

```
    while (TRUE) {
```

```
        down(&customers);
```

```
        down(&mutex);
```

```
        waiting = waiting - 1;
```

```
        up(&barbers);
```

```
        up(&mutex);
```

```
        cut_hair( );
```

```
    }
```

```
}
```

```
void customer(void)
```

```
{
```

```
    down(&mutex);
```

```
    if (waiting < CHAIRS) {
```

```
        waiting = waiting + 1;
```

```
        up(&customers);
```

```
        up(&mutex);
```

```
        down(&barbers);
```

```
        get_haircut( );
```

```
    } else {
```

```
        up(&mutex);
```

```
    }
```

```
}
```

```
/* # chairs for waiting customers */
```

```
/* use your imagination */
```

```
/* # of customers waiting for service */
```

```
/* # of barbers waiting for customers */
```

```
/* for mutual exclusion */
```

```
/* customers are waiting (not being cut) */
```

```
/* go to sleep if # of customers is 0 */
```

```
/* acquire access to 'waiting' */
```

```
/* decrement count of waiting customers */
```

```
/* one barber is now ready to cut hair */
```

```
/* release 'waiting' */
```

```
/* cut hair (outside critical region) */
```

```
/* enter critical region */
```

```
/* if there are no free chairs, leave */
```

```
/* increment count of waiting customers */
```

```
/* wake up barber if necessary */
```

```
/* release access to 'waiting' */
```

```
/* go to sleep if # of free barbers is 0 */
```

```
/* be seated and be serviced */
```

```
/* shop is full; do not wait */
```

Reference

Modern Operating Systems (2nd Edition)
by Andrew S. Tanenbaum